

Musterlösung — Übungsklausur 7

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. CNNs anfällig für adversarielle Perturbationen?

✓ (b) Hohe Dimensionalität erlaubt fein abgestimmte Muster

- (a) Nur Texturfeatures
- (c) Keine Residual Connections
- (d) Zu wenig Training

2. Max Pooling vs. Average Pooling?

✓ (b) Max Pooling: lokal translationsinvariant — Maximum bleibt im Fenster erhalten

- (a) Weniger Parameter (beide keine)
- (c) Average ignoriert schwache Akt.
- (d) Rechengeschwindigkeit

3. Batch vs. Online Sparse Coding?

✓ (b) Batch: W + alle Codes in einer Iteration; Online: zufälliges Sample, W approximativ

- (a) Umgekehrt
- (c) Äquivalent
- (d) Online nur kleines N

4. Freie Energie cRBM?

✓ (b) Energie nach Marginalisierung über h — entspricht 2-schichtigem NN

- (a) Sichtbare Energie
- (c) Normalisierungskonstante
- (d) Trainings-Loss

5. Stateless vs. Stateful Moving Average?

✓ (b) Stateless: begrenzter Horizont (Schiebefenster). Stateful: unbegrenzter Horizont mit internem Zustand

- (a) Umgekehrt
- (c) Äquivalent
- (d) Stateful mehr Parameter

6. Predictive Uncertainty (Ensemble)?

✓ (b) Diskrepanz der Vorhersagen zwischen Modellen (Standardabweichung der Outputs)

- (a) Mittelwert
- (c) Maximaler Loss
- (d) Gewichtsvarianz

7. Globale Effekte: BMs vs. GMMs?

✓ (b) GMMs: lokale Dichtebereiche. BMs: Produkt von Faktoren, jeder erfasst globalen Effekt

- (a) Beide global
- (c) BMs effizienter
- (d) BMs nur linear

8. Shapley Values Approximation?

✓ (b) Approximativ durch Mittelung über T zufällig gewählte Permutationen

- (a) Exakt über 2^d Subsets
- (c) Via Gradienten
- (d) Via LRP

2 Theorie — Multi-Task Learning

(a) Kombierter Loss für L=3 Aufgaben

$$\mathcal{L}(y, t) = \sum_{j=1}^3 \mathcal{L}_j(y_j, t_j)$$

(b) Regression + Klassifikation

$$\begin{aligned}\mathcal{L}_{\text{total}} &= \mathcal{L}_{\text{regression}}(y_1, t_1) + \lambda \cdot \mathcal{L}_{\text{classification}}(y_2, t_2) \\ &= (y_1 - t_1)^2 + \lambda \cdot \log(1 + \exp(-y_2 \cdot t_2))\end{aligned}$$

(c) Herausforderungen

Problem: Tasks können unterschiedliche Skalen und Schwierigkeiten haben. Dominanter Task verhindert Lernen des anderen.

Lösung 1: Gewichte λ_j als Hyperparameter per Validation Set tunen. Lösung 2:

Gradientennormalisierung (GradNorm). Lösung 3: Uncertainty-basiertes Gewichten (Kendall et al.) — Tasks mit hoher Unsicherheit werden geringer gewichtet.

3 Theorie — Pathologische Gradienten in RNNs

(a) Lineares RNN als Matrixgleichung

$$(y_i, h_i) \mathbf{A} = (A \ B; C \ D) \cdot (x_i, h_{i-1}) \mathbf{A}$$

(b) Exploding/Vanishing Gradienten

Gradienten fließen durch viele Zeitschritte $\rightarrow$ Produkt von Jacobi-Matrizen.

Größter Eigenwert λ_{max} der versteckten Gewichtsmatrix > 1 : Produkt wächst exponentiell $\rightarrow$ Exploding Gradients.

$\lambda_{\text{max}} < 1$: Produkt fällt exponentiell $\rightarrow$ Vanishing Gradients. Beides verhindert effektives Lernen von Long-Range-Dependencies.

(c) Gegenmaßnahmen

1. Gradient Clipping: Skaliere Gradienten, wenn Norm einen Schwellwert überschreitet. Verhindert Exploding Gradients.
2. Rauschen auf Hidden States: Zwingt Netz, sich gegenüber unteren Schichten zu desensibilisieren $\rightarrow$ reduziert Exploding Gradient Risiko.

4 Theorie — Predictive Uncertainty

(a) Ansatz 1: Explizite Kodierung

Netz produziert zwei Outputs: μ (Vorhersage) und σ (Unsicherheit). Loss: negative log-Likelihood unter Gaußannahme:

$$\mathcal{L}(\mu, \sigma, t) = (t - \mu)^2 / (2\sigma^2) + \log(\sqrt{2\pi} \cdot \sigma)$$

Hohes $\sigma \rightarrow$ hohe Unsicherheit $\rightarrow$ Loss-Beitrag kleiner, aber $\log(\sigma)$ bestraft zu große σ .

(b) Ansatz 2: Ensemble

Trainiere M verschiedene Netze (verschiedene Initialisierung). μ = Mittelwert der Outputs. σ = Standardabweichung der Outputs. Hohe $\sigma \rightarrow$ Modelle sind uneins $\rightarrow$ hohe Unsicherheit.

(c) Zusammenhang MDN

MDN ist eine Verallgemeinerung von Ansatz 1: statt einer Gaußkomponente werden M Gaußkomponenten verwendet. Damit können multimodale bedingte Verteilungen modelliert werden — wichtig bei inversen Problemen wo mehrere Outputs gleich wahrscheinlich sind.

5 Programmierung — ResNet Block und CNN Ausgabegröße

(a) Ausgabegröße berechnen

Nach Conv2d(3,16,5,stride=1,padding=0): $H_{\text{out}} = (32-5)/1+1 = 28$. Ausgabe: $16 \times 28 \times 28$.

Nach MaxPool2d(2,stride=2): $H_{\text{out}} = (28-2)/2+1 = 14$. Ausgabe: $16 \times 14 \times 14$.

(b) Residual Block

```
class ResBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(64, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 64, 3, padding=1), nn.BatchNorm2d(64))
        self.relu = nn.ReLU()
    def forward(self, x):
        return self.relu(self.block(x) + x) # Skip Connection
```